

Administrator Guide

Running an SD Core for Windows installation: accounts, security, remote access and the machine.

SD CORE FOR WINDOWS W1.0-0

Copyright and licence

Who wrote this set, how it is licensed, and what the licence lets you do with it.

This page carries the copyright and the licence for **the whole Administrator set**. Every other page in the set names them in one line at the foot; the full terms are here, once, so that they are stated in one place and cannot come to disagree with each other.

RELEASED WITH	SD Core for Windows W1.0-0
COPYRIGHT	Copyright © 2026 Donald Montaine
LICENCE	Creative Commons Attribution-ShareAlike 4.0 International (CC BY-SA 4.0)

You are free to **share** this document - copy and redistribute it in any medium or format - and to **adapt** it - remix, transform and build upon it - for any purpose, including commercially.

Two conditions apply. **Attribution:** you must give appropriate credit, provide a link to the licence, and indicate if changes were made. **ShareAlike:** if you remix, transform or build upon this document, you must distribute what you produce under the same licence. You may not add legal terms or technological measures that restrict others from doing anything the licence permits.

This is a summary and not a substitute for the licence itself. The complete text is at <https://creativecommons.org/licenses/by-sa/4.0/>.

The other two sets

The documentation is three separate sets, and each is handed out on its own: **GettingStarted**, **User** and **Administrator**. Each carries its own copy of this page, under the same copyright and the same licence.

This set is the one that can be withheld. Everything in it describes verbs an ordinary account does not have and cannot run, so an administrator may hand out the other two and keep this one. That is why it links to nothing outside itself: a reader given only the other sets must never land on a page that is not there.

The set this page belongs to

Read Accounts and Security for the verbs that decide who may use an installation and what they may do with it.

Accounts and Security

Making and changing accounts, passwords, and who may enter an account.

These are the verbs that decide **who may use this installation and what they may do with it**. They are the administrator's set, they are the smallest set on the machine, and almost all of them need more than the verb before they will do anything.

This document is separate so that it can be withheld. Everything in the administrator set describes verbs an ordinary account does not have and cannot run. It is a complete set on its own and **links to nothing outside itself**, so that handing somebody the user documentation without this never leaves them at a page that is not there. Where a user-set page is worth naming, it is named in words rather than linked.

SD folds case, so a command may be typed in either case. Commands are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

Read this before anything else: there are two gates, not one

Having the verb is not having the right to use it.

the tier	decides whether the account's VOC has the verb at all. Only an <code>administrator</code> account has these names; in any other account they are not recognised
-----------------	---

elevation	decides whether the verb does anything. Almost every verb here begins by testing it, and stops if the session is not elevated
------------------	---

In an administrator account, from an ordinary unelevated session:

```
:list.grants don
Command requires administrator privileges
:delete.account sdsys
Command requires administrator privileges
:modify.password sdsys
Command requires administrator privileges
```

Every one of those refusals came before the command was looked at. `delete.account sdsys` would have been refused anyway — you cannot delete `SDSYS` — and it never got that far.

What counts as elevated is a Windows question and is fixed when SD starts. The token a process gets is decided at process creation, so there is nothing you can type inside a running session to elevate it. **Start SD from an elevated terminal**, or accept the UAC prompt that `logto sdsys` raises. The three verbs on this page that do *not* test elevation are `clean.account`, `config` and `update.accounts`, and each of them acts only on the account you are already standing in or on your own session.

Making an account: `create.account`

```
create.account user <name> {administrator | programmer}
                        <ssh | api | both | none> {no.query}

create.account group <name> {no.query}

create.account other <name> <pathname> {no.query}
```

<code>user</code>	an SD account and a Windows local account to sign in as
-------------------	--

<code>group</code>	a shared workspace with no Windows account, reached only with <code>logto</code>
--------------------	--

<code>other</code>	an SD account over a directory you name
--------------------	---

One of `ssh`, `api`, `both`, `none` is required for a user account and there is no default. An account meant to be reached only with `logto` says `none` and means it. An `administrator` account is given both routes and takes no keyword. Omitting the tier gives a **standard** account.

It prompts for a password and `no.query` does not suppress that. `no.query` covers the confirmation, not the credential — a password is never an argument anywhere in SD. `create.account user` **therefore cannot be driven from a script**, and a group account, which has no password, can.

`sh-on` and `os-on` may be added to give the new account the `sh` verb and `OS.EXECUTE`; see `modify.account` below, which has both and their `-off` forms.

THE ACCESS MODEL IN FOUR SENTENCES

Administrators have API access and `OS.EXECUTE` access automatically. Programmers and standard users do not, unless it is given specifically. **API access does not give `OS.EXECUTE` access. And an administrator's ssh and API access stops at this machine.**

The fourth sentence is not a grant and there is no keyword for it. An administrator keeps both routes permanently — they cannot be taken away — and neither reaches past this computer: a sign-in from anywhere else is refused after the password has been checked.

An administrator may not sign in to this machine from another one.

Administration happens at the console, or through a remote desktop or remote-control product installed as a service, because those are the sessions Windows can show a consent prompt on. **Only the administrator tier is affected**; programmers and standard users reach the machine remotely on whatever routes they were given.

The third sentence is the one a reader will otherwise get wrong, because the two grants look like one grant. They are not:

<code>api</code>	says who may connect
<code>os-on</code>	says who may reach the operating system

Holding the first has never implied the second. They are independent settings and are checked in different places.

Neither can be taken away from an administrator. `modify.account` refuses `os-off` and `sh-off` for an administrator account, exactly as it refuses `ssh`, `api` and `none`.

WHAT OS-ON ACTUALLY COSTS

Granting `os-on` to a non-administrator is a larger decision than it looks.

That account reaches the operating system over the API as well as locally — and over the API the session's process token is LocalSystem. On a programmer-tier account over a remote API connection, the operating system reports the session as `nt authority\system`.

This is accepted behaviour rather than a defect, and it follows from the way SD's listener creates a session. It is stated here because it is not something a reader should have to discover: `os-on` on a non-administrator account with API access gives that account the operating system, as LocalSystem, from any machine that can reach the API port.

The full account of what creating a user account does to Windows — the groups, the disabled-then-enabled login, the console denial — is in the SD Core for Windows release documentation, under *Account types*, and is not repeated here.

Changing one: `modify.account`

```
modify.account account standard | programmer | administrator | suspended
modify.account account standard | programmer ssh | api | both | none
modify.account account ssh | api | both | none
modify.account account sh-on | sh-off
modify.account account os-on | os-off
modify.account account add | delete user.name
```

THE TIER MOVES IN ANY DIRECTION

`standard`, `programmer`, `administrator` and `suspended` are four names for one state, and any of them can be reached from any other with no intermediate step. The account's VOC is rewritten there and then, not at its next login.

`suspended` denies access and changes nothing else. The VOC is left as it is, no Windows group membership moves, and the tier it displaced is remembered so that the way back knows where the account is coming from. **There is no `resume` keyword** — coming back names the destination tier, and naming one on a suspended account lifts the suspension into it.

Three doors refuse a suspended account: signing in, `logto`, and the API.

Leaving `administrator` takes three things away and one must be named. Windows `Administrators` membership and the operating-system access record go by themselves, because the account held them *because* it was an administrator. **ssh and the API were a rule and now have to be a choice**, so the second form above is compulsory: `modify.account don programmer both`.

THE OTHER KEYWORDS

<code>ssh</code> <code>api</code> <code>both</code> <code>none</code>	what the account's remote access is . Absolute: <code>api</code> takes ssh away
---	--

<code>sh-on</code> <code>sh-off</code>	the <code>sh</code> verb and <code>!</code>
--	---

<code>os-on</code> <code>os-off</code>	<code>OS.EXECUTE</code> and the screen editors
--	--

<code>add</code> <code>delete</code> <i>user</i>	put a Windows user in, or out of, the account's group
--	---

The first group is absolute and the second is not. `ssh`/`api`/`both`/`none` are four names for one state, so whatever is not named is withdrawn. `sh-on` and `os-on` are two independent switches over two separate settings, so `sh-off` says nothing about `OS.EXECUTE` and leaves it alone.

All of these refuse an administrator account. An administrator has full access and there is no way to turn any of it off — the way to reduce one is to move it to another tier, which is what the tier keywords are for. A group account is refused too: it has no Windows user to put in a group.

Passwords: `modify.password`

```
modify.password {account}
```

With no account name it changes **your own**, and asks for the current password first. With one it changes somebody else's, which needs elevation and does not ask for the old password — an administrator resetting a forgotten password does not know it.

The password is never an argument, and a trailing token is refused rather than ignored:

```
:modify.password don hunter2
```

A password is never given on the command line; MODIFY.PASSWORD prompts for it

That refusal is the point of the verb's design. The older behaviour set the password from the prompt and threw the extra word away without a word, so every visible sign said it had worked — while the password had already reached SD's command stack, and a shell's history and process list if the verb was reached from one. Refusing does not put it back, but it says so.

The prompts are hidden, asked twice, and the account must already exist in the register. `modify.password` **cannot be scripted**, by design.

Who may enter an account: `grant`, `revoke`, `list.grants`

```
grant account to user
revoke account from user
list.grants account
grant account          the same as list.grants
```

The grant is Windows group membership and nothing is written to the account record. Every SD account has a Windows group — `sdu_name` for a user account, `sdg_name` for a group account — and entry to the account **is** membership of that group. These three verbs edit that group and read it back.

<i>There is no Windows account named %1</i>	<code>grant</code> to somebody who does not exist
<i>%1 is already a member of group %2</i>	<code>grant</code> to somebody who already has it
<i>%1 has not been granted account %2</i>	<code>revoke</code> from somebody who has not
<i>Account not registered in ACCOUNTS file</i>	the account name is not one of SD's
<i>Account %1 has no Windows group recorded</i>	a record predating the group model; it is refused, not guessed at

The account's own user is listed like any other member. It is not a grant — creating the account put them there — but hiding it would make the listing disagree with the Windows group it is reporting.

A successful `grant` or `revoke` writes an audit record to SD's audit trail, stamped with the user and process that did it. The identity is stamped by SD and is not passed in, so a caller cannot get it wrong or forge it. Windows records the group change in its own security log as well, and the two are independent records of the same act.

`modify.account` **account** `add / delete` **user makes the same group change and writes no audit record.** Prefer `grant` and `revoke` when there is a choice: they say what they mean and they leave a trail.

Continued in

Account Maintenance — `clean.account`, `update.accounts`, `[locked]`, `config`, `set.date` and `delete.account`.

Account Maintenance

Emptying scratch files, refreshing a VOC, keeping local versions of records, configuration, the system date, and deleting an account.

This page continues Accounts and Security.

Emptying an account's scratch files: `clean.account`

```
clean.account
```

Empties three things in the account you are standing in, and takes no arguments — **there is no way to clean an account you are not in:**

```
:clean.account  
Cleaned $COMO  
Cleaned $hold  
Cleaned $savedlists
```

<code>\$COMO</code>	captured session transcripts, including every phantom's
<code>\$hold</code>	reports sent to the hold file instead of a printer
<code>\$savedlists</code>	saved select lists

Nothing else is touched — no data file, no program, no dictionary. A como capture that is currently running is left alone and says so: *\$COMO not cleaned - COMO file active*.

It needs no elevation. It is the one verb here an administrator account can use from an ordinary session, which is right: it deletes only that account's own scratch.

Refreshing an account's VOC: `update.accounts`

```
update.accounts {all}
```

```
Copying records from NEWVOC to VOC...
```

Copies the shipped verb and keyword definitions into an account, adding what is missing and leaving that account's own VOC entries alone. It is what brings an existing account up to date after SD itself is upgraded, and **it respects the account's tier** — a standard account does not collect programmer verbs by being refreshed, and a suspended one keeps the tier it was suspended from.

With no keyword it updates the account you are standing in and then offers the rest, asking each time. `all` is **the unattended form**: it updates every registered account without asking.

You will not normally type `all` yourself. The installer runs it during an upgrade, so a release that adds a VOC record reaches every existing account without anybody visiting them. Before that existed, an upgrade replaced the shipped files and no account gained a new verb.

`all` is an explicit keyword rather than something inferred from how SD was started. The test for an internal session exists and would have worked, but it would have decided a rewrite of every account's VOC from a property nobody typing the verb can see.

A second word that is not `all` is refused by name rather than ignored:

```
:update.accounts everything  
UPDATE.ACCOUNTS does not take everything
```

Quietly ignoring it would run the interactive form while the caller believed they had asked for the other one.

IT NEVER TAKES ANYTHING AWAY

`update.accounts` only ever adds. A record removed from an account stays removed. To keep an edit of your own, mark the record — see below.

Keeping your own version of a VOC record: `[locked]`

This has no equivalent in OpenQM or in SD on Linux. Nothing you know from another MultiValue system will tell you it exists.

A site that has customised one of SD's own VOC records marks it by putting `[locked]` in **field 1, after the type code**. `update.accounts` then leaves that record alone.

```
V[locked]
CA
$MYVERSION
```

AFTER THE TYPE CODE, NOT AT THE FRONT

The first character of field 1 **is** the type, so `[locked]V` would be read as a record of type `[ ]`. Two characters are the type for a `[P]` record, and those are the only two-character types SD uses: `[PA]` for a paragraph and `[PH]` for a phrase.

```
PA[locked]
```

Anywhere later in field 1 works — the test searches the field rather than matching a fixed position. Case does not matter: both sides are upper-cased before comparison, so `[LOCKED]` and `[Locked]` are the same marker. A lock that failed open because somebody typed it in capitals would be worse than no lock at all, because the record it was meant to protect would be replaced silently.

A VERB IS NOT PROTECTED BY IT

`[locked]` **is honoured on every kind of VOC record except a verb**. A verb marked `[locked]` is updated anyway, and the account is told which ones:

```
2 verb(s) marked [locked] were updated anyway: MYLIST MYREPORT
```

A verb is what SD runs. A locked one would go on naming the program, or the internal routine number, that this release replaced — and the internal case fails in the worst way available, because field 3 of an internal verb record is a *number*, so a reassignment sends the verb to a different function with no sign that anything is wrong.

The way to get a verb that behaves differently is to add your own, under a name SD does not ship, modelled on the system one. `update.accounts` walks the records SD ships, so a verb of your own is never visited and needs no marker.

YOU ARE TOLD WHAT WAS WITHHELD

```
3 VOC record(s) were left alone because they are marked [locked]: WINDOWS ...
```

The message names each record, and says plainly that any correction this release made to them has not been applied here.

That is the cost, and it belongs to the site rather than to SD. A locked record keeps the version the account already had, so a defect fixed in that record stays unfixed in this account. Remove the marker from field 1 and run `update.accounts` again to take the new version.

Reading and setting configuration: `config`

<code>config</code>	report every setting
<code>config lptr</code>	the same, to the default printer
<code>config param value</code>	set one
<code>config gpl</code>	display the licence
<code>config contrib</code>	display the contributors

```
:config
Virtual Machine Version Number W1.0-0
APILOGIN 1
APIPORT 4243
CMDSTACK 99
DEADLOCK 0
DUMPDIR
ERRLOG 50 kb
...
NUMFILES 80
NUMLOCKS 100
NUMUSERS 20
...
SH C:/Windows/System32/WindowsPowerShell/v1.0/powershell.exe -NoProfile -NoLogo
SORTWORK /cygdrive/c/WINDOWS/TEMP
TEMPDIR /cygdrive/c/WINDOWS/TEMP
YEARBASE 1930
```

(Forty-odd lines; cut here.) **Reading needs nothing** — any account with the verb can do it, and it is the quickest answer to *how many users, how many locks, how many open files is this machine set up for*.

New parameter value required

`config numlocks` with nothing after it. **The report form is `config` alone**; naming one parameter means *set it*

Not a recognised private configuration parameter name

the name is not one that can be set per-session

Invalid value for this parameter

it is, and the value is not

`config param value` sets a **private, session-local value and not the machine's**. The machine's settings live in SD's configuration file and are read when SD starts. This form overrides one for the session you are in, which is the right tool for trying a value before writing it down and the wrong one for changing an installation.

`config gpl` and `config contrib` read a record inside SD rather than running a pager over a file, so they work in any account and need no operating-system access.

Setting the machine's date: `set.date`

```
set.date date
```

Sets the **machine's** date, not a session preference — it changes the clock the whole installation reads. The argument goes through SD's `D` conversion, so anything `iconv(..., 'D')` accepts will do, and anything it does not is refused:

Date required

`set.date` with nothing after it

Invalid date format

the argument is not a date SD can read

There is no confirmation and no undo. It is described here from source rather than shown running, because demonstrating it would move the clock of whatever machine it ran on. **On Windows, changing the system date is itself a privileged operation**, so a session that has the verb may still be refused by the operating system underneath it.

Moving a live machine's date backwards is not a neutral act: file timestamps, licence expiry, scheduled tasks and anything that reasons about elapsed time all read it. Treat it as a maintenance operation on a quiet system.

Deleting an account: `delete.account`

```
delete.account account.name
```

Removes the account directory, its Windows group, its entry in the accounts register, and — for a user account SD itself created — the Windows account and its profile. **One confirmation covers all of it**, and the wording is decided before the question is asked, so it never offers to remove a Windows account it is not going to.

It will not delete a Windows account SD did not create. The account is left in place and it says so.

Three refusals come before the confirmation, so none of them can be reached by accident:

Cannot delete SDSYS account

`delete.account sdsys`

Cannot delete own account

the account you are standing in

Account not registered in ACCOUNTS file

the name is not one of SD's

(Those three wordings are the verb's own; they are not shown as a transcript here because reaching them takes an elevated session, and an unelevated one is refused by the privilege gate first — which is itself the fourth refusal, and the one most people meet.)

The confirmation is unconditional and no keyword suppresses it. There is no `no.query` on this verb. **Never send `delete.account` down a pipe** — the prompt will eat the commands that follow it as its answers, and the session will then wait for ever.

Who has these verbs

All of them are administrator verbs. A standard or programmer account has none of these names at all.

needs elevation as well

`create.account` `modify.account` `modify.password` (for another account)
`delete.account` `grant` `revoke` `list.grants`

the verb is enough

`clean.account` `update.accounts` `config`

`list.grants` **needs elevation even though it only reads.** It answers *who may enter this account*, which is worth knowing before you have it, and the gate is at the top of the program the three grant verbs share.

See also

Sessions and Locks · Operating System Access.

In the user documentation, which does not repeat any of this: *SD TCL - The Command Processor* for how a verb is dispatched and what a VOC record holds, and *SD Basic - System and Environment* for what a program can read about its own session. Those pages are in a different set and are deliberately not linked from here — see the note at the top.

Sessions and Locks

Seeing who is logged in, ending a session that will not end itself, and inspecting or forcing the locks a session holds.

These are the verbs for looking at the machine as a whole and intervening in it: **whose sessions are running, what they are holding, and how to take either away**. Every one of them is administrator-tier, and the ones that act on somebody else need an elevated session on top of that.

This document is separate so that it can be withheld. It links to nothing outside the administrator set. Where a user-set page is worth naming, it is named in words.

SD folds case, so a command may be typed in either case. Commands are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

Every listing on this page was produced by running it, on SD Core for Windows W1.0-0. The lock listings come from a program that took the locks first and then ran the verb against itself, because a lock nobody holds is not something a listing can show.

Two things are not on this page, on purpose

A programmer needs to understand locks without being able to force one, so the *user* documentation keeps the part of the subject that is theirs: what a task lock is against a database lock, and `release`, which gives back locks your own session holds and needs no privilege at all. The same goes for `status`, `phantom`, `pstat`, `pdebug` and `pdump` — a programmer's view of their own processes.

This page is the other half: the verbs that see and act across the machine.

Who is logged in: `listu`

```
listu {no.page} {lptr {n}}
```

```
:listu
  User  Pid          Puid  Login time    Origin : Username
    12  1789          27  Aug 13:14    : don
    19  1808          27  Aug 13:53    : don
  *   27  1828          27  Aug 13:59    : don
```

***** the session you are typing in

User SD's user number — what `logout`, `pstat` and `pdump` all take

Pid the Windows process id

Puid the user number of the **parent**, filled in for a phantom and blank otherwise

Origin where the session came from

Username the **Windows** account the session runs as, not the SD account it is in

Username is the column that matters for permission. Almost every control on this page is keyed to the Windows identity, not to the SD account — so two sessions showing different accounts but the same username are, as far as `logout` is concerned, the same person's.

Origin is blank for a local session and that is not a fault. It reports an address or a device name, and a console or piped session has neither. It reads `Phantom` for a phantom and an IP address for a network client.

`SDNet` is also among the values it can print, and you will not see it: SDNet was removed from this port, so nothing can open the kind of session that would report it.

`(logout pending)` after the name means somebody has asked that session to end and it has not gone. See below.

Ending a session: `logout`

```
logout                end this session
logout n {n ...}      end another session, by user number
logout all             end every session but this one
```

`logout` with no argument ends your own session. It is `quit` under another name, which is worth knowing before typing it intending to list something.

```
:logout 999
```

Only administrators can logout processes running with other usernames

A session may end only sessions running under the same Windows account unless it is elevated. The refusal above came from an unelevated session, and user 999 does not exist — **the privilege test happens before the number is looked up**, so this message does not tell you whether the user was real.

`logout all` is stricter still: elevated **and** run from the `SDSYS` account. It leaves your own session alone.

WHEN A SESSION WILL NOT END

`logout` **signals** a process. If the process has already gone — killed from outside SD, or lost with a terminal — there is nothing to signal, and the entry stays with `(logout pending)` beside it.

The entry matters because it holds a slot and an exclusive-access claim. `NUMUSERS` counts it, and any verb that wants a file to itself — `build.index` is the usual one — is refused while it is there. **Recovery is not another** `logout`:

```
sd -cleanup
```

run elevated, and a restart of the SD service if that does not take it.

Confirm the session is actually dead before clearing it. `pstat n` answers (*Not responding*) for a session with nothing behind it — it asks the process and waits about four seconds — and that is the difference between a dead entry and a busy one. `pstat` is a programmer verb and is documented in the user set.

What is locked: `list.readu`

```
list.readu {user.no} {detail} {wait} {no.page} {lptr {n}}
```

```
:list.readu
```

There are no active file, read or update locks held by any user

That answer is about the **whole system**, not about you. With locks held:

```
User File Path..... Type Id.....
  23   1 /cygdrive/c/ProgramData/SD/user RU   R1
      _accounts/don/ZZLK31A
```

```

19      4 /cygdrive/c/ProgramData/SD/user RU    zzlock31
        _accounts/don/voc
23      1 /cygdrive/c/ProgramData/SD/user RL    R2
        _accounts/don/ZZLK31A

```

User the SD user number holding it — `listu` turns that into a person

File SD's internal file number, and **this is the number** `unlock` wants

Path wrapped over as many lines as it needs, in POSIX form

Type `RU` update · `RL` read · `FX` exclusive file lock · `SX` shared file lock · `WAIT` a session waiting for one

Id the record id; blank for a file lock

A file lock has no id:

```

User File Path..... Type Id.....
 23    1 /cygdrive/c/ProgramData/SD/user FX
        _accounts/don/ZZLK31A

```

The PATH is the POSIX one and that is not a display fault. SD holds file paths internally in `/cygdrive/c/...` form on this port. It names the same place as `C:\ProgramData\SD\...`.

THE KEYWORDS

`detail` prints the record-lock budget above the listing:

```

:list.readu detail
Record lock limit (NUMLOCKS) = 100, Current = 0, Peak = 2
There are no active file, read or update locks held by any user

```

Peak is the one to watch. `Current` describes this instant, which is rarely the instant that mattered. A peak near the limit is how you find out `NUMLOCKS` needs raising **before** a program fails with the lock table full. `config` reports the configured limit.

A user number restricts the listing to one session. `wait` adds the sessions *waiting* for a lock, as `WAIT` rows — left out by default, so the plain listing shows the cause of a hold-up and not its victims. **Ask for** `wait` **when something is stuck.**

A LOCK OUTLIVING ITS SESSION

The `19 ... RU zzlock31` row above is real, and it is a lock on a record in an account's `voc` file left behind by a session that was killed while holding it. **A dead session's locks are not released**, any more than its user-table entry is, and everything wanting that record waits for a process that is not there.

Take the user number to `pstat`. (*Not responding*) means clear the session rather than wait for it.

Task locks: `list.locks`, `lock`, `clear.locks`

Task locks are **64 numbered flags with no connection to any data**. They mean whatever the programs using them agree they mean — usually *only one session runs this job at a time*, where the thing being protected is not a single file.

```
list.locks
lock n {no.wait}
clear.locks {n}
```

```
:list.locks
No task locks reserved by any user
```

and with lock 5 held by user 16, `*` marking your own session:

```
:list.locks
0:      1:      2:      3:      4:      5: 16*    6:      7:
8:      9:     10:     11:     12:     13:      14:     15:
16:     17:     18:     19:     20:     21:     22:     23:
```

(64 numbers over eight rows; the rest are blank and are cut here.)

`LOCK` WAITS UNLESS YOU TELL IT NOT TO

```
:lock 5 no.wait
Set task lock 5
:lock 5 no.wait
Task lock already owned by this process
```

Without `no.wait` **it waits for ever.** It prints *Waiting for task lock to become available* once, then retries every two seconds until it gets it — right for a job that must run, wrong for anything unattended. `no.wait` **turns the wait into a refusal** (*Task lock is already in use*), which a script can act on.

`CLEAR.LOCKS` GIVES BACK YOUR OWN, AND ONLY YOUR OWN

```
:clear.locks 5
Released task lock 5
:clear.locks
All task locks released
```

With no number it releases all 64 this session holds, and says *all* whether it held any or not.

<i>Released task lock n</i>	it was yours and it is now free
-----------------------------	---------------------------------

<i>Task lock n is held by another process</i>	<code>clear.locks</code> will not take it
---	--

<i>Task lock n is not held by any process</i>	it was already free
---	---------------------

<i>Task lock number must be in range 0 to 63</i>	from <code>clear.locks 99</code>
--	----------------------------------

Forcing a lock open: `unlock`

```
unlock file n {user n} record.id {record.id ...}
unlock file n {user n} all
unlock file n {user n} filelock
unlock tasklock n {n ...}
```

This is the only verb that takes somebody else's lock, and it needs an elevated session — having the verb is not enough:

```
:unlock
Command requires administrator privileges
```

It names a file by number, not by name — the `File` column of `list.readu`. That is deliberate: the lock is on a file the machine has open, which may not be in your VOC at all and may be reached under different names in different accounts.

<code>file</code> <i>n</i>	the file, by the number <code>list.readu</code> printed
<code>user</code> <i>n</i>	restrict to one session's locks
<code>all</code>	every record lock matching, rather than named ids
<code>filelock</code>	the file lock, which cannot be combined with record ids
<code>tasklock</code> <i>n</i>	force a task lock, including one held by another process

A file number or a user number is compulsory — *Either a file number or a user number must be specified* — so there is no `unlock` that means *everything on the machine*.

Unlocking is not free and SD cannot make it so. A lock is a promise its holder is relying on. Forcing one open while its owner is alive lets two sessions write the same record and tells neither. **Establish that the holder is dead first** — `pstat` on the user number, `listu` for (*logout pending*) — and prefer clearing the session to clearing the lock.

`UNLOCK TASKLOCK` IS THE ONLY WAY BACK FROM A KILLED HOLDER

Task locks are released when a session ends normally. They are not part of any file, so nothing has to be written back.

`sd -cleanup` **does not give them back, and that is a defect.** It releases a dead session's record locks and file locks and leaves its task locks held, by a user number nothing is behind, until SD itself is restarted. `list.locks` shows the number with an owner and `clear.locks` refuses it because it is not yours. **`unlock tasklock n`, elevated, is the way out** — that is what the forced form is for. It is recorded in the project's fix lists.

Who has these verbs

All of them are administrator-tier, so a standard or programmer account does not have the names at all.

the verb is enough `listu` `list.readu` `list.locks` `lock` `clear.locks`

needs elevation as well `unlock`, and `logout` against another Windows account's session

The split is between looking and intervening. Seeing who is on the machine and what they hold is an administrator's ordinary business. Taking a session or a lock away from somebody else asks for the elevated token as well.

See also

Accounts and Security · Operating System Access.

Operating System Access

The `sh` and `!` verbs, the list that decides who may use them, and why the list is keyed to the person rather than the account.

`sh` runs a Windows command from the SD prompt. `!` is the same verb under a shorter name. **They are the only way out of SD to the operating system from TCL**, and who may use them is decided by a file in the system account rather than by an account's tier.

This document is separate so that it can be withheld. It links to nothing outside the administrator set. Where a user-set page is worth naming, it is named in words.

SD folds case, so a command may be typed in either case. Commands are shown here in lower case.

Every listing on this page was produced by running it, on SD Core for Windows W1.0-0, from an unelevated session in an account that is on the list.

The two verbs

```
sh command
! command
```

Everything after the verb is handed to the shell as typed:

```
:sh echo hello-from-the-shell
hello-from-the-shell
:! echo via-the-bang-form
via-the-bang-form
```

Never type `sh` with nothing after it in a script. The configured shell for the bare form is interactive — `config` reports it as

```
SH          C:/Windows/System32/WindowsPowerShell/v1.0/powershell.exe -NoProfile -NoLogo
```

```
SH1 C:/Windows/System32/WindowsPowerShell/v1.0/powershell.exe -NoProfile -NonInteractive
```

so a bare `sh` in a piped session, a phantom or a scheduled job hands control to a shell with nobody at the keyboard and **waits for ever**. The `SH1` form, which is what `sh` command uses, is `-NonInteractive`.

The shell you get is Windows PowerShell 5.1

Not `pwsh`, and the difference bites:

```
:sh echo delta && echo epsilon
At line:1 char:12
+ echo delta && echo epsilon
+                ~~
The token '&&' is not a valid statement separator in this version.
```

`&&` and `||` **do not exist there**. Nor do the ternary, null-coalescing or null-conditional operators. Chain with `;`, and test with `if ($?) { ... }`.

Pipes and redirection do work, for an account that is on the list:

```
:sh echo alpha-beta | findstr alpha
alpha-beta
:sh echo gamma > zzsh.txt
:sh Get-Content zzsh.txt
gamma
```

The second and third lines are separate `sh` invocations, so **the working directory persists between them** — the file written by one was read by the next.

SD refuses to nest

```
:sh sd
SD is already running in this session - type EXIT to return to it.
```

The child shell is marked, and SD checks the mark:

```
:sh Get-ChildItem Env:SD_SESSION
Name                               Value
----                               -
SD_SESSION                         1
```

That is worth knowing when writing a script for `sh` to run — anything it invokes inherits `SD_SESSION`, so a script that starts SD as part of its work will be refused, wherever it is called from.

Who is allowed: `os.users`

The permission is a record in a file in the system account, not a tier. `sdsys/os.users` holds one record per person:

record id the **Windows login name**

field 1 ☐ `yes` to allow `sh` and `!`

field 2 ☐ `yes` to allow `OS.EXECUTE` from a program, and the screen editors

Anything other than ☐ `yes` means no, and **a missing file or a missing record means no**. That is the opposite of the tier lists, where a missing record means the full set — do not carry the convention across.

It is keyed to `@logname`, the person, not the account. The permission therefore does not change when somebody `logto`'s somewhere else. That is deliberate: the question *may this person reach the operating system* has one answer per person, and an account they can enter should not be able to change it.

EDITING IT

`os.users` is an ordinary SD file in the system account, edited with `ed` from `SDSYS` — which needs an elevated session to enter. `create.account` **writes the record for an administrator account as it creates it**, with both fields ☐ `yes`, and `modify.account`'s `sh-on`/`sh-off`/`os-on`/`os-off` keywords set the two fields afterwards. Hand-editing remains the only route for the case those keywords refuse.

The file's ACL is the whole of the protection. `os.users` is read-only to `sdusers` on disk, which is what stops somebody adding their own name to the list from inside their own account. The command processor reads it in the user's own process. **Without that ACL this control is decoration** — if you are hardening an installation, check it rather than assume it.

The three outcomes

The gate and the metacharacter rule are separate tests, and they combine like this:

	PLAIN COMMAND	PIPES, REDIRECTION, CHAINING
on the list	runs	runs
not listed, elevated	runs	refused 5240
not listed, unelevated	refused 10053	refused 10053

don is not permitted to use the operating system shell

is message 10053, and it names the person rather than the account.

The middle row is the one people misread. An elevated session that is not on the list keeps a restricted shell: it may run a command, but not one containing shell metacharacters. **Being on the list is what buys a real shell** — pipes, redirection and chaining are most of what a programmer wants one for, and that was the ruling behind lifting the ban for listed accounts. Elevation on its own does not lift it.

What this does not gate

A program's `OS.EXECUTE` is not on this path. `OS.EXECUTE` from SD BASIC compiles to a different opcode and is governed by field **2** of the same record, not by field 1 and not by the metacharacter test. So the two halves are set independently, and `sh-off` deliberately leaves `OS.EXECUTE` alone.

The form that does go through this gate is `execute 'sh ...'` from a program, because that runs the TCL verb.

The screen editors read field 2, not field 1 — an account can be refused `sh` and still run `edit` and `micro`, and the reverse.

Who has these verbs

`sh` and `!` are administrator-tier, so an ordinary account does not have the names. **And the tier is not the permission** — an administrator account whose Windows login is not in `os.users`, and whose session is not elevated, has the verb and is refused by it. **Two gates, and both must pass.**

An SD administrator is a shell on this machine

State this plainly to anyone deciding who gets an administrator account:

An SD administrator can run operating-system commands on the server as LocalSystem — from this machine. A sign-in from any other computer is refused, over ssh and over the API alike.

The local half is not a defect, and no single setting produces it. It follows from three rules that are each reasonable on their own:

An administrator always has API access	and it cannot be taken away
An administrator always has <code>OS.EXECUTE</code>	and that cannot be taken away either
For a session that arrived over a socket, <code>os.users</code> is the authority	the session's own token is LocalSystem

So an administrator account is, in effect, an operating-system shell on the server, and the operating system reports such a session as `nt authority\system`.

WHY REMOTE IS SHUT

Those three rules would have made the administrator tier a shell for *anyone who could reach the port*, from anywhere. SD closes that at the door instead of weakening any of the three: **administration requires a session** **Windows can show a consent prompt on**, which means the console, or a remote-desktop or remote-control product installed as a service — not ssh and not the API.

The refusal comes *after* the password has been checked, so it is a refusal rather than a silent drop, and it says what it is:

An administrator may not sign in to this machine from another one.

Only the **tier** is refused. An ordinary or programmer account reaches the same machine over the same route and gets a session as before, which is worth knowing when you are diagnosing a connection that failed: if a non-administrator can get in, the network and the listener are fine.

THE ONE ROUTE THIS DOES NOT CLOSE

An ssh tunnel ends on the server, so a connection forwarded through one arrives looking local, and is admitted. SD sees the address the connection came from, and a tunnelled connection genuinely comes from this machine.

That is a real limit and it is stated here rather than glossed over. Building such a tunnel still needs an ssh login to Windows in the first place, so it is not open to a stranger — but if your threat model includes an administrator who should not be administering remotely, close port forwarding in `sshd_config` rather than relying on this gate.

What limits the rest of it is who holds an administrator account, who holds a credential for one, and who can reach the console.

The verify suite asserts all of this — the local session working, the remote one refused, and an ordinary account admitted over that same remote route as the control — so a future change that quietly reopened it shows up as a failing test rather than as a page that had silently become false.

The first logto out of the system account ends elevation

An administrator working in `SDSYS` is elevated. **The moment they `logto` anywhere else, that elevation is given up** — the administrator flag is cleared and the elevated helper is stopped.

This is deliberate. Without it the helper would outlive the rights it belongs to, leaving a session able to do privileged work from an ordinary account.

The consequence a reader meets first is that **a second hop is refused**:

```
:logto sales
:logto payroll
```

The second `logto` is not running as an administrator any more, so it costs a fresh UAC prompt. Recover by going back:

```
:logto sdsys
```

An administrator who chains two hops and does not know this will read it as a fault. It is the design working.

The step is quiet for the ordinary case: a session moving between two ordinary accounts never had privilege, so nothing is given up and nothing is written to the audit trail.

See also

Accounts and Security · Sessions and Locks · Remote Access and the Machine.

Encryption and the SEXT interface

What libsodium provides, what an application can reach, and why field encryption has no key route in W1.0-0.

SD Core for Windows links libsodium and ships it as `libsodium-26.dll` beside the server. Two things are built on it: the credential exchange that authenticates an API login, and a pair of BASIC functions that encrypt and decrypt a string.

This document is separate so that it can be withheld. It links to nothing outside the administrator set. Where a page in another set is worth naming, it is named in words.

SD folds case, so a command may be typed in either case. Commands are shown here in lower case.

The short version

An administrator reading this page usually wants one of two answers.

For credentials, nothing needs configuring. Passwords are never stored. SD keeps a SCRAM-SHA-256 verifier in the credential store, the API login proves knowledge of the password without sending it, and the primitives behind that exchange are the ones listed further down.

For encrypting application data, W1.0-0 does not provide a usable route. The functions exist and work, but the only way to produce a key they accept is an internal-only call. This is set out in full under *Why field encryption is not available* below, because a site planning to encrypt fields needs to know before it writes the application, not after.

What was removed

The `encrypt.field` verb

Removed. It pointed at a program, `$CRYPTO`, which never existed in the GPL release, so the verb could not have worked in any build derived from it

<code>encrypt()</code> and <code>decrypt()</code>	Removed upstream in July 2024 and replaced by <code>sdencrypt()</code> and <code>sddecrypt()</code> . The old names do not compile
The Python half of STEXT	Removed with the embedded interpreter. The <code>SD_Py*</code> keys, <code>SDPYFUNC.H</code> , twenty <code>PY_*</code> programs and the object opcode all went. A program referencing a Python key does not compile, and the state it tested for cannot be reached

sdencrypt() and sddecrypt()

These are ordinary BASIC functions. Any account may compile a call to them.

```
sdencrypt(data, key, encoding)
sddecrypt(data, key, encoding)
```

Three arguments, not two. The third selects how the key is encoded and how the result is returned:

ENCODING	MEANING	KEY LENGTH REQUIRED
<code>201</code>	hexadecimal	64 characters
<code>202</code>	base64	44 characters

The cipher is libsodium's authenticated `secretbox`, so the key is exactly 256 bits. The two lengths above are that key after encoding, and they are checked exactly: a key of any other length is refused.

WHY FIELD ENCRYPTION IS NOT AVAILABLE

A passphrase is not a key, and this is where a reader will otherwise lose a day. On W1.0-0:

```
sdencrypt('The quick brown fox', 'secretkey', 202)
```

returned nothing and set `status()` to **10204**, a key length error. `secretkey` is nine characters and the function wanted 44.

The function that turns a password into a key of the right length is `sdext()`'s `SD_KEYFROMPW`, and `sdext()` is internal-only — it needs a program compiled with `$internal`, which needs an administrator working in the system account. **So an ordinary program cannot obtain a key these functions will accept, and there is no supported way in.**

An application that must encrypt data in W1.0-0 should do it outside SD, in the client, and store the result as an ordinary string.

The SDEXT interface

`sdept()` is the internal entry point to the cryptographic primitives.

```
rtntal = sdept(arg, isargmv, key)
```

`key` selects the operation, `arg` carries the arguments and `isargmv` says whether `arg` holds several values rather than one.

It cannot be called from an application. `sdept` is in the compiler's internal function table, so an ordinary account does not merely get refused — it gets a misleading error. An unknown function is read as a matrix reference, so the complaint is about a `dim` statement the program does not contain, reported at the last line rather than at the call. That behaviour is covered by SD Basic - Restricted Commands in this set.

`$internal` needs both halves: the compiler tests for internal mode **and** for the administrator flag. Internal mode alone was enough until 13 August 2026 and was not safe, because internal programs are the only ones that may set the administrator flag.

THE KEYS

Ten are implemented. Every binary value is base64, because the interface carries NUL-terminated strings and a raw 32-byte digest would contain a mark character about one time in nine.

KEY	VALUE	ARGUMENTS	RESULT
<code>SDEXT_TestIt</code>	1	any	Prints each argument and returns a count. A diagnostic
<code>SD_SALT</code>	100	none	A fresh salt, base64
<code>SD_KEYFROMPW</code>	101	password, salt	A 256-bit key derived from the password
<code>SD_EUID_SET</code>	102	user name	Sets the process effective user and group
<code>SD_EUID_RESTORE</code>	103	none	Restores what they were on entry
<code>SD_SHA256</code>	104	one	SHA-256 of the argument
<code>SD_HMACSHA256</code>	105	base64 key, text message	HMAC-SHA256

KEY	VALUE	ARGUMENTS	RESULT
<code>SD_PBKDF2</code>	106	password, salt, iterations, length	Derived key
<code>SD_RANDBYTES</code>	107	count	Random bytes
<code>SD_XORBYTES</code>	108	two equal-length values	Their exclusive-or
<code>SD_CTEQUAL</code>	109	two values	<code>1</code> or <code>0</code> , compared in constant time

`SD_CTEQUAL` reports a malformed argument as an error rather than as `0`, because by the time the login path compares these values both sides are the server's own — a decode failure there is a defect, not a wrong password. A caller deciding whether to admit a login must still treat the error as a refusal.

`SD_EUID_SET` and `SD_EUID_RESTORE` call the POSIX identity functions provided by the server's runtime. They do not change the Windows process token, and SD's own identity model does not use them.

WHAT USES IT

Eight shipped programs call `sdxext()`, and they are the whole of its use: `APISRV`, `CRED_SET`, `CRED_VERIFY`, `SD_GET_SALT`, `SD_KEY_FROM_PW`, `SDCLIENT`, `EUID_SET` and `EUID_RESTORE`. Between them they set a credential, verify one, and run the API's SCRAM exchange.

None of the eight is catalogued for general use, and none has a VOC entry, so they are not an indirect route to the interface either.

Two constants that are not STEXT keys

`SD_ENCODEHX` (201) and `SD_ENCODE64` (202) are defined alongside the STEXT keys and are easily mistaken for them. They are not keys and `sdxext()` does not implement them — passing either as a key returns a key error. They are the third argument to `sdencrypt()` and `sddecrypt()`, and they are documented above.

Remote access and the machine

The four verbs that change Windows rather than SD, and the one thing they all require.

Most administrator verbs change something inside SD. These four change the machine SD is running on: whether an ssh server exists, who may reach it, whether SD opens its API socket, and whether `sd` runs from any directory.

<code>ssh.server</code>	adds or removes the Windows OpenSSH server
<code>remote.ssh</code>	decides who may reach it
<code>remote.api</code>	decides whether SD opens its API socket, and who may reach it
<code>append.sd.path</code>	puts SD's program directory on the Windows system PATH, or takes it off

None of these gives an administrator remote access, and no setting does. Opening ssh or the API to other computers opens it for ordinary and programmer accounts. **An administrator account is refused a session from any machine but this one**, over either route, after its password has been checked — so a route you have just opened will still turn your own administrator account away if you try it from your laptop. That is deliberate: administration needs a session Windows can show a consent prompt on, which means the console, or a remote desktop or remote-control product installed as a service. The operating system access page covers why the tier is treated this way.

This document is separate so that it can be withheld. It links to nothing outside the administrator set. Where a page in another set is worth naming, it is named in words.

SD folds case, so a command may be typed in either case. Commands are shown here in lower case.

All four need an elevated session, including to report

Each of them begins by testing the administrator flag, and stops if it is not set:

```
:ssh.server  
Command requires administrator privileges
```

That was an administrator account. Holding the administrator tier is not enough — **the session itself has to be elevated**, and the test runs before the keyword is read, so the reporting forms are refused too.

The reason is the work they do rather than the information they give. Each one reaches Windows through SD's elevated helper, and an unelevated session has no helper to reach it through. Reading the answer costs the same rights as changing it: `Get-WindowsCapability` cannot report whether a Windows capability is installed without administrator rights either.

An administrator gets an elevated session by signing in at the console as a Windows administrator, which elevates at login. A session that reached an ordinary account by `logto` does not have one, and cannot get one — the first `logto` out of the system account ends elevation deliberately, which is covered under operating system access in this set.

Every one of them reports when given no keyword

```
ssh.server  
remote.ssh  
remote.api  
append.sd.path
```

With no keyword each reports the current state and changes nothing. This is the supported way to answer "how is this machine set up?", and for ssh it is the only reliable way — see below.

ssh.server

```
ssh.server {install | remove}
```

FINDING OUT WHETHER A SERVER IS INSTALLED

Windows 11 does not list the OpenSSH server in Settings → Apps, and it has never appeared in Control Panel → Programs and Features. Somebody looking there concludes it was never installed, which is the wrong conclusion and an easy one to reach.

It is a Feature on Demand, not a program. In Windows 11 it lives under **Settings → System → Optional features**; in Windows 10 it was under Settings → Apps, which is where anyone who learned this on Windows 10 will look first.

The PowerShell route is not a fallback either, because `Get-WindowsCapability -Online` needs administrator rights even to read.

So `ssh.server` with no keyword is the answer: it reports, changes nothing, and works because it goes through SD's elevated helper.

The OpenSSH server is installed and running.

Who may reach it is a separate setting - use "remote.ssh" to see or change it.

INSTALL

The capability comes from Windows Update. It can take several minutes, and up to about an hour on a slow machine or connection, with little on screen while it runs. The verb asks before starting for that reason.

Afterwards Windows needs a restart before the service exists, so no SD account can sign in until the machine has been restarted.

REMOVE

`remove` asks first, and the question is not a formality. Accounts SD creates sign in over ssh or over the API and are denied the console and Remote Desktop. For an account that has only ssh — including at this keyboard, where it arrives by `ssh localhost` — removing the server takes away its only way in.

The removal is staged behind a reboot. Windows reports success while `sshd.exe` is still on disk and the service is still running, so ssh continuing to work afterwards is expected and is not a fault.

One consequence is worth knowing before you remove it. `C:\ProgramData\ssh` is left in place, because Windows does not remove it with the capability. It holds the host keys and `sshd_config`. That matters for one thing: running the SD installer on this machine again. Setup compares `sshd_config` against the copy Windows ships, `sshd_config_default` — and that copy went with the capability, so Setup cannot tell whether the configuration has been edited and stops rather than guess.

`ssh.server install` is not affected: it puts the server back and `sshd_config_default` comes back with it. Remove the directory as well only if you do not intend to run an ssh server on this machine again.

remote.ssh

```
remote.ssh {on | off}
```

<code>on</code>	other computers on the network may connect over ssh
-----------------	---

<code>off</code>	only this computer may connect
------------------	--------------------------------

It takes effect at once, needs no restart, and disconnects nobody.

`off` **does not stop the server.** It scopes the Windows firewall rule `OpenSSH-Server-In-TCP` rather than disabling it, and that distinction is load-bearing: disabling the rule would also break `ssh localhost`, which is how an SD account signs in at this keyboard. To remove the server itself, use `ssh.server remove`.

If there is no ssh server, the verb says so and points at `ssh.server install` rather than reporting a state it cannot have.

remote.api

```
remote.api {on | local | off}
```

<code>on</code>	SD listens on port 4243 and other computers may connect
-----------------	---

<code>local</code>	SD listens on port 4243 and only this computer may connect
--------------------	--

<code>off</code>	SD opens no API socket at all
------------------	-------------------------------

There are two axes here and the verb sets both. `APIPORT` in `sd.conf` decides whether SD opens a socket at all; the firewall rule decides who may reach it. `local` exists because they are separate — listener up, firewall shut.

That is also why the three keywords behave differently:

<code>on</code> and <code>off</code>	change whether SD opens a socket, which it decides when it starts. They offer to restart SD
--------------------------------------	---

<code>local</code> and <code>on</code>	differ only in the firewall when the listener is already up, and take effect at once
--	--

The restart ends every SD session, including the one that asked for it. The verb says so in the question rather than afterwards. Declining leaves the setting written and it takes effect when SD is next started.

If the listener is set but the firewall rule cannot be written, the verb says which half succeeded and prints the two commands that set the rule by hand. This is a real state and not a theoretical one: the setting and the rule are different objects and either can fail on its own.

append.sd.path

```
append.sd.path {on | off}
```

on	put SD's program directory on the Windows system PATH
----	---

off	take it off
-----	-------------

This is what lets `sd` run from any directory rather than only its own.

It is the system PATH, not the user's. The value lives under `HKLM\...\Session Manager\Environment`, so the change applies to every account on the machine. A terminal that is already open keeps the PATH it started with; open a new one to see the change.

Turning it off does not make SD unreachable. `sd` still runs from its own directory or by full path, and `append.sd.path on` puts it back.

WHY THIS VERB EXISTS

It is not symmetry with the other three. The installer's PATH tickbox could set this once and never again: nothing removed the entry except the uninstaller, so clearing the box on a later run did nothing at all. An upgrade shows no tasks page whatsoever, on the principle that an administrator who wants to make further choices has command-line tools for them — and that left the PATH setting with no way to be changed after the first install. This is that tool.

WHY IT IS NOT CALLED SET.PATH

Inside SD, *path* already means the **account's** path: `pathname` is a VOC keyword, `where` prints the account pathname, and `@path` holds it. `set.path` would read as setting that. `os.path` collides with the notion of a host filesystem path. `append.sd.path` cannot be read as either.

When a change fails

All four report a failure rather than falling silent, and every failure message names what did not happen and confirms that nothing was changed. Two status values recur and mean different things:

status 1	Windows refused the change, or SD is not running elevated
----------	---

status -1 the helper could not be run at all

`remote.api` adds status 2, which means `sd.conf` could not be read — missing, unreadable, or carrying no `APIPORT` line to recognise.

System limits

The sizes, counts and depths compiled into SD Core for Windows, and which of them a site can change.

Two kinds of number appear on this page and they behave differently.

A **compiled limit** is fixed in the build. Changing it means rebuilding SD, and several would change on-disk file formats, so in practice they are ceilings.

A **configured limit** is a value in `sd.conf` with a default. A site can raise or lower it, within a range the compiled limits set. Those are covered in full under *Configuration* in this set; this page gives the number in force and says which parameter controls it.

This document is separate so that it can be withheld. It links to nothing outside the administrator set. Where a page in another set is worth naming, it is named in words.

Every figure here was read from the sources that define it, or from a running W1.0-0 system, and not from documentation of another product.

Data

Maximum string or record size	1,073,741,822 bytes	compiled
Maximum record id length	63 by default, up to 255	MAXIDLEN
Fields per record	limited only by record size	—
Values per field, subvalues per value	limited only by record size	—

The record size limit is just under 1 GB rather than 2 GB. It is the maximum size of any single string SD will build, so it applies to a record, to a concatenation, and to anything a program assembles in memory.

A record id may be longer than 63 characters only if `MAXIDLEN` is raised, and the range starts at 63 rather than 0 — a lower value is refused at start-up rather than accepted. The ceiling of 255 is compiled, and the comment in the source notes that raising it would require major file-format changes.

Files

File types	two — directory and dynamic	compiled
Group size for a dynamic file	2 KB by default, maximum 8	<code>GRPSIZE</code>
Open files across all sessions	80	<code>NUMFILES</code>
Maximum pathname length	255	compiled
Sort keys in one query	32	compiled

Group size is expressed in 1 KB units, so the compiled maximum of 8 is 8 KB.

Sessions and locks

Concurrent sessions	20	<code>NUMUSERS</code>
Record locks across all sessions	100	<code>NUMLOCKS</code>
File locks	one per file	compiled
Windows account name	32 characters	compiled
SD account name	32 characters	compiled

`NUMUSERS` sizes the user table in shared memory, so it is read once when SD starts and cannot be changed in a running system.

Programs

Subroutine call depth	10,000	<code>MAXCALL</code> , range 10 to 1,000,000
Catalogued name length	63 characters	compiled

Matrix dimensions	two	compiled
Compiled programs held in memory	no limit by default	<code>OBJECTS</code>
Memory those programs may occupy	no limit by default	<code>OBJMEM</code> , in KB

Select lists

Numbered select lists	13 , numbered 0 to 12	compiled
Items in a select list	limited by disk	—

Thirteen is the number a session has available at once, not a limit on how many lists can be saved. `save.list` writes to a file and is limited only by space.

Terminals

Terminal definitions shipped	63 source definitions	—
Terminal names they compile to	100	—
Default page size	120 × 36	—
Minimum page size	10 lines × 20 columns	compiled
Default printer page	80 × 66	<code>LPTRWIDE</code> , <code>LPTRHIGH</code>

The extra terminal names are aliases and variants that share a definition with their base name.

What each account tier gets

These are inventories rather than limits, and they are the numbers most often misquoted, so they are given here with what they count.

	VERBS	VOC RECORDS
Standard	82	355
Programmer	124	397
Administrator	147	420

The two columns count different things. A VOC record may be a verb, a keyword, a file pointer, a sentence or a paragraph; only some are verbs. A reader asking "how many commands can this account type?" wants the first column. A tool comparing two accounts with `count voc` sees the second.

Both are computed from the shipped VOC rather than maintained by hand: the verb figures come from the same two tier lists the account-creation code reads, and the record figures are what `count voc` reports in a freshly created account.

Configuration

Parameters accepted in <code>sd.conf</code>	52
Parameters the <code>config</code> verb displays	43
Parameters readable with <code>config()</code>	49
Parameters settable for the current session	28
Parameters accepted that do nothing	6

The six inert parameters, and why each is inert, are listed under *Configuration* in this set. `NETFILES` and `CREATUSR` are the two a site is most likely to have in an existing `sd.conf`.

Configuration

The sd.conf file, the two ways to read a setting, and every parameter SD accepts.

SD reads one configuration file at start-up. It sizes the shared memory segment, sets the limits every session inherits, and names the directories SD writes to.

This document is separate so that it can be withheld. It links to nothing outside the administrator set. Where a page in another set is worth naming, it is named in words.

SD folds case, so a command may be typed in either case. Commands are shown here in lower case.

Every listing on this page was produced by running it, on SD Core for Windows W1.0-0, from an unelevated session in an administrator account.

The file

C:\ProgramData\SD\sd.conf

The server and the client both read the `SD_CONFIG` environment variable first and fall back to that path. The file is installed only if it does not already exist and is marked never to uninstall, so edits to it survive an upgrade and survive removal of the product.

It is plain text in one section:

```
[sd]
SDSYS=C:\ProgramData\SD\sdsys
GRPSIZE=2
NUMUSERS=20
```

Lines beginning `#` are comments. The shipped file is heavily commented and those comments record why each value was chosen. Read them before changing anything.

A NAME SD DOES NOT RECOGNISE STOPS IT STARTING

An unrecognised parameter is not ignored. The parser abandons the file with `Unrecognised configuration parameter`, and SD does not start.

That is why obsolete names are still parsed rather than deleted. `CREATUSR` has done nothing since 14 August 2026, but a configuration file copied from a Linux installation still carries it, and removing the branch would turn a tidy-up into a failure to start.

Range failures behave the same way. `GRPSIZE`, `INTPREC`, `LPTRHIGH`, `LPTRWIDE`, `MAXCALL`, `RECCACHE`, `SORTMRG` and `MAXIDLEN` are bounds-checked once the file has been read, and a value outside its range stops start-up with a message naming the parameter.

Reading the settings

The `config` verb reports what is in force:

```
:config
Virtual Machine Version Number W1.0-0
APILOGIN  1
APIPORT   4243
CMDSTACK  99
DEADLOCK  0
DUMPDIR   C:\ProgramData\SD\sdsys\dumps
ERRLOG    50 kb
...
YEARBASE  1930
```

SD accepts **52** parameters and the verb prints **43** of them. Nine are accepted in the file and never displayed:

`CODEPAGE`, `CREATUSR`, `DEBUG`, `FDS`, `FIXUSERS`, `NETDIRS`, `PORTMAP`, `SDSYS` and `TXCHAR`.

`NETDIRS` is the one to know about. It decides what an API session may reach outside its own account, and the verb will not tell you what it is set to. Read it from the file.

The `config()` function reads one parameter from a program:

```
group.size = config('GRPSIZE')
```

49 parameters are readable this way. The three that are not are `CREATUSR`, which stores nothing, `SDSYS`, and `TXCHAR`. The name is truncated to eight characters before it is matched; no parameter name is longer than eight, so that only shows up if you pass something which is not a parameter.

Two further forms exist. `config lpnr` reports the settings of the default printer, and `config gpl` and `config contrib` display the licence and the list of contributors.

Changing a parameter

Editing `sd.conf` and restarting SD is the durable route, and for most parameters it is the only one.

28 parameters can also be changed for the current session:

```
config sortmem 8192
```

That writes to the session's own copy of the settings, taken from shared memory when the session started. It does not reach `sd.conf`, it does not affect any other session, and it is gone when the session ends.

The 28 are `CODEPAGE`, `DUMPDIR`, `EXCLREM`, `FILERULE`, `FLTDIFF`, `FSYNC`, `GDI`, `GRPSIZE`, `INTPREC`, `LPTRHIGH`, `LPTRWIDE`, `MAXCALL`, `MUSTLOCK`, `OBJECTS`, `OBJMEM`, `RECCACHE`, `RINGWAIT`, `SAFEDIR`, `SDCLIENT`, `SH`, `SH1`, `SORTMEM`, `SORTMRG`, `SORTWORK`, `SPOOLER`, `TEMPDIR`, `TERMINFO` and `YEARBASE`.

Everything else takes effect only when SD is next started. That includes every limit which sizes the shared memory segment and every setting the API listener reads.

Sessions and limits

PARAMETER	DEFAULT	EFFECT
<code>NUMUSERS</code>	20	Maximum concurrent sessions. Sizes the user table in shared memory
<code>NUMFILES</code>	80	Maximum open files across all sessions
<code>NUMLOCKS</code>	100	Maximum record locks across all sessions
<code>MAXCALL</code>	10000	Maximum subroutine call depth. Range 10 to 1000000
<code>CMDSTACK</code>	99	Depth of the command stack
<code>FDS</code>	unset	Limit on file descriptors. No limit when unset
<code>FIXUSERS</code>	unset	<code>base,range</code> — user numbers reserved for sessions that ask for a specific number

PARAMETER	DEFAULT	EFFECT
PORTMAP	unset	<code>base_port,base_user,range</code> — gives a session a fixed user number derived from the port its connection arrived on. Refused if the range overlaps <code>FIXUSERS</code>

Files and locking

PARAMETER	DEFAULT	EFFECT
GRPSIZE	2	Default group size for a new dynamic file, in 1 KB units. Read by <code>create.file</code> and <code>configure.file</code> when no group size is given
MAXIDLEN	63	Maximum record id length. 63 is also the lower bound
MUSTLOCK	0	When 1, a <code>write</code> or <code>delete</code> requires the record to be locked first
DEADLOCK	0	When 1, SD traps deadlocks
SAFEDIR	0	When 1, directory files are updated by write-and-rename rather than in place
RECCACHE	0	Records cached per file. Range 0 to 32
FSYNC	0	Bit flags controlling when SD forces data to disk
FILERULE	0	Bit flags deciding which special VOC file references are honoured. Bit 4 allows a <code>PATH:</code> reference to name a pathname directly

Directories

PARAMETER	DEFAULT ON A NEW INSTALL	EFFECT
SDSYS	<code>C:\ProgramData\SD\sdsys</code>	The system account. SD does not start if the global catalogue is not found beneath it
USRDIR	<code>C:\ProgramData\SD\user_accounts</code>	Where <code>create.account</code> puts a user account
GRPDIR	<code>C:\ProgramData\SD\group_accounts</code>	Where <code>create.account</code> puts a group account
DUMPDIR	<code>C:\ProgramData\SD\sdsys\dumps</code>	Where process dumps are written

PARAMETER	DEFAULT ON A NEW INSTALL	EFFECT
TEMPDIR	/cygdrive/c/WINDOWS/TEMP	Temporary files
SORTWORK	/cygdrive/c/WINDOWS/TEMP	Work files for a sort that does not fit in memory
JNLDIR	empty	Journal directory
TERMINFO	empty	An additional terminfo directory. The shipped definitions are found without it

DUMPDIR is set rather than left empty on purpose. A blank value falls back to the system directory, which SD users can write to, and a process dump carries the whole variable state of the session that wrote it. The installer makes the dump directory write-only to SD users, so a dump can be added and nobody else's can be listed or read.

TEMPDIR and SORTWORK are reported in POSIX form because that is how the server's runtime addresses them. C:\WINDOWS\TEMP and /cygdrive/c/WINDOWS/TEMP are the same directory.

The API

PARAMETER	DEFAULT	EFFECT
APIPORT	4243	The port the API listens on. If the line is absent no socket is created at all, which is how the API is turned off
APILOGIN	1	Whether the API requires authentication. 0 is the weaker setting, not the safer one
NETDIRS	unset	Directories outside its own account an API session may open, separated by semicolons because a Windows path contains a colon
SDCLIENT	0	Restricts what an API session may do. Non-zero disables file access outright; 2 additionally refuses any subroutine not compiled as callable from a client

Unset is the strict value for NETDIRS. With nothing there, an API session can open files in the account it is standing in and nothing else. It never grants the credential store, the global catalogue, os.users or the account register, and naming those has no effect. A directory listed here is reachable by every API session in every account, so it is a decision about the machine rather than about one account.

SDCLIENT is not reported by the config verb and has no entry in the shipped sd.conf. It defaults to 0, which permits everything.

Printing

PARAMETER	DEFAULT	EFFECT
<code>LPTRWIDE</code>	80	Default page width for the printer. Range 10 to 1000
<code>LPTRHIGH</code>	66	Default page depth for the printer. Range 10 to 32767
<code>SPOOLER</code>	empty	The spooler to hand print jobs to
<code>GDI</code>	0	Selects the Windows printing interface used by default

Sorting

PARAMETER	DEFAULT	EFFECT
<code>SORTMEM</code>	4096 kb	Above this much data a sort works on disk instead of in memory
<code>SORTMRG</code>	4	Files merged at once in a disk sort. Range 2 to 10

Numbers and dates

PARAMETER	DEFAULT	EFFECT
<code>INTPREC</code>	13	Digits of precision in integer arithmetic. Range 0 to 14
<code>FLTDIFF</code>	0.00000000002910	Two floating point numbers closer together than this compare equal
<code>YEARBASE</code>	1930	The century a two-digit year is read into

Diagnostics

PARAMETER	DEFAULT	EFFECT
<code>ERRLOG</code>	50 kb	Size of the error log. A non-zero value below 10 KB is raised to 10 KB, and the oldest entries are discarded when it fills
<code>PDUMP</code>	0	Bit flags controlling when a process dump is written
<code>DEBUG</code>	unset	Bit flags enabling debugging features
<code>STARTUP</code>	empty	A command run when SD starts
<code>JNLMODE</code>	0	Journalling mode
<code>OBJECTS</code>	0 (no limit)	Compiled programs held in memory at once
<code>OBJMEM</code>	0 (no limit)	Memory those programs may occupy, in KB

The shell

PARAMETER	DEFAULT	EFFECT
<code>SH</code>	PowerShell, interactive	The shell a bare <code>sh</code> starts
<code>SH1</code>	PowerShell, non-interactive	The shell <code>sh command</code> uses

Both are full paths on a real install:

<code>SH</code>	<code>C:/Windows/System32/WindowsPowerShell/v1.0/powershell.exe -NoProfile -NoLogo</code>
<code>SH1</code>	<code>C:/Windows/System32/WindowsPowerShell/v1.0/powershell.exe -NoProfile -NonInteractive</code>

The difference between them matters. `SH1` carries `-NonInteractive` and `SH` does not, so a bare `sh` in a phantom or a scheduled job hands control to a shell with nobody at the keyboard. Operating system access has its own page in this set.

Parameters that are accepted and do nothing

These are parsed so that an existing `sd.conf` still loads. None of them changes SD's behaviour, and none should be offered to a site as a control.

PARAMETER	WHY IT IS INERT
-----------	-----------------

NETFILES	SDNet was removed from this port. <code>netfiles.c</code> is deleted, a <code>server;file</code> VOC reference is refused, and the request that reports open SDNet connections returns an empty list. The value is still stored and still reported, and setting it opens nothing
CREATUSR	SD has accounts rather than accounts and users, so <code>create.account</code> always creates the operating system account and there is nothing to opt in to. The value is discarded as it is read
CODEPAGE	Stored, readable and settable. Nothing acts on it
EXCLREM	Stored, readable and settable. It described exclusive access to a remote file, and there are no remote files
RINGWAIT	Stored, readable and settable. Nothing acts on it
TXCHAR	Stored, and not readable — there is no <code>config('TXCHAR')</code>

`FILERULE` is **not** in this list, and is easy to mistake for it. Its remote bits died with SDNet, but bit 4 is live and decides whether a `PATH:` VOC reference may name a pathname directly.

There is no licence parameter and no licence verb. SD Core for Windows is GPL v3 and is not licensed per site or per user; `config gpl` displays the licence text.

Installation and the service

What the installer puts on the machine, what an upgrade replaces, and how the service behaves.

This page covers the parts of installation an administrator has to live with afterwards: the groups and permissions the installer creates, the service, and what an upgrade does and does not touch.

This document is separate so that it can be withheld. It links to nothing outside the administrator set. Where a page in another set is worth naming, it is named in words.

The wizard itself — the pages, the tick boxes and their defaults — is covered in the SD Core for Windows release documentation, under *Installing SD Core*, and is not repeated here.

Two things to know before the first install

SD cannot be installed silently. `/SILENT` and `/VERYSILENT` are refused with a message rather than ignored.

The reason is the password. Installing ends by asking for one, and an unattended install has nobody to ask — it would finish with no password on any account, and an account without a password cannot be used at all: not at the keyboard, not over ssh, and not through the API. A `/NOPASSWORD` escape was written and then removed, on the grounds that a switch buying a credential-less system is a switch somebody will paste from a forum.

Remote Desktop is not unattended and is unaffected. The wizard runs and a person answers it.

The installer refuses to start on a machine with an ssh server it does not own. It changes nothing and tells you what it found. Three cases:

- another ssh server holding port 22
- an ssh service that is not part of Windows, installed even if it is not running
- Windows' own ssh server with settings somebody has already changed

The check runs before the wizard is drawn, so a refusal costs nothing. Remove the other server, or put the ssh configuration back the way Windows shipped it, and run the installer again.

This is why SD can make promises about ssh behaviour at all: it configures the server so that SD sessions land inside SD and cannot reach a command prompt, and it can only promise that about a server it installed and configured itself.

What lands where

WHAT	WHERE
Binaries, and the MSYS2 DLLs beside them	C:\Program Files\SD\usr\bin\
The client DLLs, for applications	C:\Program Files\SD\usr\clients\
The installed PowerShell scripts	C:\Program Files\SD\
The changelog	C:\Program Files\SD\changelog
Configuration	C:\ProgramData\SD\sd.conf
The SDSYS account	C:\ProgramData\SD\sdsys\
User accounts	C:\ProgramData\SD\user_accounts\
Group accounts	C:\ProgramData\SD\group_accounts\
Shared memory	C:\ProgramData\SD\shm\

Neither root can be changed. The installer does not ask.

Do not move the binaries. `usr\bin` is load-bearing. Shipping `msys-2.0.dll` beside the executable relocates the POSIX root to the DLL's directory minus two components, so moving them moves SD's idea of the file system.

The client DLLs are the one thing under `Program Files` meant to be copied elsewhere. `usr\clients\client64\` holds the 64-bit pair and `usr\clients\client32\` the 32-bit pair, for applications that link against SD. The 64-bit pair also appears in `usr\bin`, and that copy is the server's own — a local client connection resolves `sd.exe` beside the DLL, so only a copy sitting next to `sd.exe` can make one.

What the installer creates

<code>sdusers</code> group	grants access to the files under <code>C:\ProgramData\SD</code>
<code>sdsshonly</code> group	carries the deny rights that confine an account to ssh
<code>sdu_<name></code>	one group per account, created by <code>create.account</code>

The service	String Database (SD) , automatic start
-------------	---

ACLs	inheritance broken on <code>C:\ProgramData\SD</code> , access granted narrowly
------	--

Group membership is carried in a Windows logon token, which is issued at sign-in. An administrator added to `sdusers` while signed in does not have it until they sign out and back in, and until then cannot read the data tree at all. The symptom looks like a broken install and is not one.

The service

Display name	String Database (SD)
--------------	-----------------------------

Service name	<code>SD</code>
--------------	-----------------

Start type	automatic — Windows starts it at every boot
------------	---

Starting	<code>Start-Service SD</code> , or <code>sd -start</code>
----------	---

Stopping	<code>Stop-Service SD</code> , or <code>sd -stop</code>
----------	---

Created by	the installer
------------	---------------

Removed by	the uninstaller
------------	-----------------

Stopping the service ends every session on the machine, without asking. It signals every entry in the user table and has no "are users logged in" check, so treat it as a machine-wide action rather than an administrative convenience.

AFTER AN UNCLEAN SHUTDOWN

A shared memory segment left behind by an abrupt stop survives a reboot on Windows. SD discards it and starts normally, saying so:

```
Discarding the shared segment left by the previous boot -  
SD did not shut down cleanly.
```

Upgrading

Installing a new release over an existing one replaces the shipped files and preserves everything the site owns.

REPLACED	PRESERVED
the catalogue and compiled programs	your accounts and their passwords
the BASIC source	the private catalogue
the messages and include records	which Windows users are linked to which SD accounts
the VOC templates and library routines	your print queue and held reports
terminfo, the licence, the contributor list	everything under your own accounts, and <code>sd.conf</code>

An upgrade asks nothing. No tasks page is shown, on the principle that the machine already carries the answers and every setting has a verb that changes it afterwards — `ssh.server`, `remote.ssh`, `remote.api` and `append.sd.path`, covered under *Remote access and the machine* in this set.

TWO STEPS RUN FOR YOU, AND BOTH REPORT

Every account's VOC is refreshed. The installer runs `update.accounts all`, which walks every registered account so that a verb this release adds can be typed in accounts that already existed.

This is newer than it sounds and is worth stating plainly: before W1.0-0 an upgrade replaced the shipped files and **no existing account, including SDSYS, ever gained a new verb**. A fix that added a VOC record reached only accounts created afterwards.

The dictionaries are reapplied. The definitions the release ships are added and updated, and any you added are left alone. If that step cannot run, the installer says so at the end rather than finishing quietly, and `upgrade-dicts.log` in `C:\ProgramData\SD` records what happened.

Neither step can take anything away. `update.accounts` only ever adds records, so an account created before a verb was withdrawn keeps it.

Uninstalling

Settings ▸ Apps, or `unins000.exe`.

The default does not touch your accounts, the database or the configuration. Removing the data is a separate prompt that defaults to keeping it.

The uninstaller does not remove OpenSSH. It may predate SD or be in use for something else. It does restore `sshd_config`, keeping the version SD wrote as `sshd_config.before-sd`.

The Installed Scripts

The thirty-seven PowerShell scripts installed beside SD - the execution policy they need, what their exit codes mean, and the ones you may need to run yourself.

SD's installer does most of its work in PowerShell rather than inside the installer script, and **it leaves every one of those scripts on the machine**. They are in `C:\Program Files\SD`, beside `sd.exe`, and they are there for two reasons: so that a step which failed during the installation can be run again without reinstalling, and so that a choice made in the wizard can be changed afterwards.

They are Windows scripts, not SD verbs. Nothing here is typed at an `sd` prompt. Run them from a PowerShell prompt started with **Run as administrator** - almost all of them refuse a prompt that is not elevated, and say so rather than half-working. `check-install.ps1` **is the exception and wants an ordinary prompt**; the section on it says why.

Italics mark something you supply, **bold** a word typed as it stands, and braces an optional part.

What is here and what is not

Thirty-seven scripts ship. They are the installer's own steps, the helpers SD launches while it is running, and the ones the administrator verbs call. Everything else in the project's `gplbld` directory - the verifiers, the probes, the build and test cycle - is development tooling and **is deliberately not installed**. If you have read about `cycle.ps1`, `assert-current.ps1` or a `verify-` script and cannot find it, that is why: they compare an install against the source tree it was built from, and they are destructive.

PowerShell execution policy — you do not need to change it

Windows will not run a PowerShell script unless its **execution policy** allows it. On Windows desktop editions the default is `Restricted`, which allows no script at all; on Windows Server it is `RemoteSigned`, which allows a script written on the machine itself.

SD does not depend on that setting, and you should leave it alone. Every one of the thirty-seven scripts is launched with an explicit `-ExecutionPolicy Bypass` on its own command line — by the installer, by the SD service, and by the SD verbs that call one. That switch applies to **that one PowerShell process, for that one script**. It changes nothing on the machine and nothing about any other script.

What happens if you change it anyway:

Tighten it to `Restricted` or `AllSigned`, for the machine or for your own account

Nothing. SD carries on working. The switch SD passes takes precedence over both of those settings

Loosen it to `RemoteSigned`, `Unrestricted` or `Bypass`

Nothing — and you have gained nothing. SD was already unaffected. You have made the machine more permissive for every *other* script on it, which is a real cost for no benefit

Set it through Group Policy

This one stops SD. See below

So if you loosened the policy to get SD working, you can put it back. That was needed on builds before 5 September 2026, where SD's own commands failed with *"running scripts is disabled on this system"*. It is fixed, and the workaround is no longer doing anything for you.

One thing it does not cover. The `sh` verb opens an ordinary PowerShell prompt for you, and that prompt gets **no** such switch — it runs under whatever policy your machine sets. That is deliberate: SD lifts the restriction for the scripts it installed itself, and never for a shell you type into.

GROUP POLICY IS THE EXCEPTION, AND IT IS THE ONE TO KNOW ABOUT

A **Group Policy** setting — *Turn on Script Execution*, under `Computer Configuration` or `User Configuration` → `Administrative Templates` → `Windows Components` → `Windows PowerShell` — **outranks the switch SD passes.** Group Policy sits above the per-process setting in PowerShell's order of precedence, so on a machine where a policy sets the execution policy, SD cannot override it.

On a domain-joined or otherwise managed machine, check this before installing. In an ordinary PowerShell prompt:

If the `MachinePolicy` or `UserPolicy` row says anything other than `Undefined`, a policy is in force. `RemoteSigned`, `Unrestricted` or `Bypass` **there is fine** — SD's scripts are written on the machine by the installer, not downloaded. **A policy of `Restricted` or `AllSigned` will stop SD's administrative commands**, and the symptom is the "*running scripts is disabled on this system*" message from `logto sdsys`, `append.sd.path`, `remote.api`, `remote.ssh` or an editor verb. **That needs your Windows administrator to relax the policy.** SD has no way around it, deliberately: a program that could defeat Group Policy would be a worse thing to have installed than an inconvenience.

The exit codes are a convention

Every script prints what it did and then exits on the same three-value convention:

-
- 0** it is done. That includes "*it was already done*" - the scripts are written to be run twice
 - 1** it failed, and the line above the exit says why
 - 2** **it neither did the work nor failed.** It refused, or it could not run, or the work needs a restart first
-

2 is the one worth reading. It is not an error code; it means the script declined to act and is telling you the condition. Each script below says what its own 2 means, because they differ - a refusal in `allow-ssh-groups.ps1` is not the same event as a pending restart in `install-ssh.ps1`. Only `sd-elevate.ps1` adds a fourth, **5**, for *not elevated*.

The ones you may need to run

These answer a question or change a decision. Every command below is complete as written; run it from an elevated PowerShell prompt.

IS THE INSTALLATION SOUND?

```
powershell -File "C:\Program Files\SD\check-install.ps1"
powershell -File "C:\Program Files\SD\check-install.ps1" -Brief
```

Exit **0** nothing is wrong, **1** something is. `-Brief` prints one line per check with no preamble. **A check that cannot be answered yet is not a failure** and it says so separately.

Run this one without elevation. It is the only script here that does, and the reason is the question it asks: *can this user's ordinary sign-in reach SD?* An administrator token reads the data tree through the `Administrators` entry on the ACL and would pass whether or not the answer is yes. The script notices it is elevated and says what the answer is worth, but that is a backstop rather than the intent. The Start Menu entry **Check the SD installation** runs it exactly this way.

And run it again later. The installer offers this check as a tick box at the end, and that run is **always the incomplete one**: the installing user's logon token cannot carry the `sdusers` group until they sign out and back in, so every database check reports "*not yet*" by design. The Start Menu entry is there so the real run can be done afterwards without writing the command down.

THE SSH SERVER WOULD NOT INSTALL

```
powershell -File "C:\Program Files\SD\install-ssh.ps1"
```

Exit **0** installed and running, **2** installed but Windows needs a restart before the service exists, **1** failed.

This is the one that matters most. Accounts SD creates sign in over ssh and nothing else, so until this succeeds nobody but you can use that SD. The installer prints this same command in its closing report when it could not install the server; it is repeated here because that report is easy to close. **It is slow** - `Add-WindowsCapability` downloads from Windows Update and can work for minutes in silence. Do not interrupt it.

AN EDITOR VERB DOES NOTHING

```
powershell -File "C:\Program Files\SD\install-editors.ps1"  
powershell -File "C:\Program Files\SD\install-editors.ps1" -CheckOnly
```

Exit **0** every editor is present, **2** at least one is missing and could not be installed, **1** failed. `-CheckOnly` reports and installs nothing. What happened last time is in `C:\ProgramData\SD\install-editors.log`.

THE SD SERVICE

```
powershell -File "C:\Program Files\SD\install-service.ps1" -Install  
powershell -File "C:\Program Files\SD\install-service.ps1" -Remove
```

Exit **0** done, **1** failed, **2** could not be attempted - not elevated, or `sdsvc.exe` is not there. `-Install` accepts `{-AppDir directory}` if SD is not in the usual place.

WHO MAY REACH THE API FROM OTHER COMPUTERS

```
powershell -File "C:\Program Files\SD\api-firewall.ps1" -Show  
powershell -File "C:\Program Files\SD\api-firewall.ps1" -Open
```

```
powershell -File "C:\Program Files\SD\api-firewall.ps1" -Restrict
powershell -File "C:\Program Files\SD\api-firewall.ps1" -Remove
```

Exit **0** applied, **1** failed, **2** refused. `-Show` changes nothing. `-Open` allows any address, `-Restrict` this machine only; add `{-Port n}` for a port other than 4243. **This script owns its rule** - it created it, and `-Remove` takes it away.

WHO MAY REACH SSH FROM OTHER COMPUTERS

```
powershell -File "C:\Program Files\SD\ssh-firewall.ps1" -Show
powershell -File "C:\Program Files\SD\ssh-firewall.ps1" -Installed -Restrict
powershell -File "C:\Program Files\SD\ssh-firewall.ps1" -Installed -Open
```

Exit **0** applied, **1** failed, **2** refused, or the rule is not there yet.

It toggles a rule it did not create. Installing the OpenSSH capability creates `OpenSSH-Server-In-TCP` and enables it for any address; this narrows it to loopback or widens it again. It has no `-Remove`, deliberately: the rule is Microsoft's and SD must not delete it.

WHO MAY SSH INTO THIS MACHINE AT ALL

```
powershell -File "C:\Program Files\SD\allow-ssh-groups.ps1" -Check
powershell -File "C:\Program Files\SD\allow-ssh-groups.ps1" -Installed
powershell -File "C:\Program Files\SD\allow-ssh-groups.ps1" -Remove
```

Exit **0** done or nothing to do, **1** failed, **2** refused.

The write needs `-Installed` and the script's own Usage text leaves it out. Without it you get *"-Installed not given"* and exit 2, and nothing is written. The switch means *an administrator asked for this*: the script rewrites `sshd_config` and restarts sshd, so it will not do that merely because it was run. `-Check` and `-Remove` do not need it.

It also refuses if `sshd_config` already says who may connect - an existing `AllowGroups`, `AllowUsers`, `DenyGroups` or `DenyUsers` line is somebody else's decision and is left alone. That refusal is also exit 2, and it prints the lines it found.

THE TWO REMOTE-ROUTE GROUPS

```
powershell -File "C:\Program Files\SD\sync-route-groups.ps1" -Check
powershell -File "C:\Program Files\SD\sync-route-groups.ps1"
```

Exit **0** success, **1** failure; it prints what it did either way. It creates the groups that decide which remote route an SD account may use, and seeds `sdssh` so an install that predates them does not lose ssh. `-Check` prints what it would do and changes nothing.

RE-STAMPING THE ACCOUNT DIRECTORIES

```
powershell -File "C:\Program Files\SD\secure-account-dirs.ps1" -Root "C:\ProgramData\SD\c  
powershell -File "C:\Program Files\SD\secure-account-dirs.ps1" -Root "C:\ProgramData\SD\c
```

Exit **0** every directory stamped, **1** at least one failed, **2** it could not run. Add `{-Account name}` for one account rather than all of them.

This is the only `secure-` **script with a reason to be run again.** The others name one fixed path and the installer has already done them; this one walks a directory whose contents grow as accounts are made, and `create.account` stamps each new account itself.

MAY SD INSTALL ON THIS MACHINE?

```
powershell -File "C:\Program Files\SD\ssh-preflight.ps1"
```

Exit **0** clear to install, **1** refuse, **2** could not determine - which is also a refusal. **It changes nothing:** it reads the service registry, one TCP table and two files. The installer runs it before the wizard is drawn, and running it yourself is how you find out in advance why an installation would be refused on a machine that already has an ssh server.

Continued in

The Scripts SD Runs For Itself — the scripts the installer, the verbs, the uninstaller and SD itself run.

The Scripts SD Runs For Itself

The scripts the installer, the administrator verbs, the uninstaller and SD itself call, which nobody types.

This page continues The Installed Scripts.

The ones the installer runs

You should not need any of these, and running one out of order can undo work rather than repeat it - most of them must run *after* the step that secures the data tree, or inheritance puts back exactly what they took away. They are listed so that a name in a log or an error message can be looked up.

<code>adopt-account.ps1</code>	gives the installing user an SD account. Without it SD installs and then refuses the person who installed it
<code>deny-logon.ps1</code>	denies a local group the console and Remote Desktop, which is what confines an account to ssh
<code>finish-install.ps1</code>	the two steps that happen after the installer closes - SD opens so you can set your own password, then the post-install check runs
<code>install-service.ps1</code>	creates, starts and removes the Windows service String Database (SD)
<code>install-editors.ps1</code>	makes sure the editors the <code>edit</code> and <code>micro</code> verbs run are on the machine
<code>ssh-preflight.ps1</code>	asks whether SD may install here at all, and refuses a machine carrying an ssh server SD does not own. Runs before the wizard is drawn
<code>sync-route-groups.ps1</code>	creates the two groups that decide which remote route an account may use, and seeds <code>sdssh</code> so an existing install does not lose ssh
<code>upgrade-dicts.ps1</code>	brings an upgraded install's dictionaries up to the release. Runs on an upgrade only
<code>upgrade-voc.ps1</code>	brings every existing account's VOC up to the release, by running <code>update.accounts all</code> . Runs on an upgrade only

<code>secure-accounts.ps1</code>	the containers account directories are created in
<code>secure-account-dirs.ps1</code>	the ACL on each account's own directory
<code>secure-audit.ps1</code>	creates the audit trail and makes it append-only
<code>secure-cred.ps1</code>	locks the credential store to SYSTEM and Administrators
<code>secure-dumps.ps1</code>	makes the process-dump directory write-only to SD users, so a dump can be added and nobody else's can be read
<code>secure-gcat.ps1</code>	locks the global catalogue, and separately the compiled objects it is loaded from
<code>secure-log.ps1</code>	creates a log only administrators can see or write
<code>secure-osusers.ps1</code>	locks a permission list so only an administrator can change who is on it. The installer calls it four times - twice for <code>os.users</code> and twice for <code>batch.jobs</code>
<code>secure-pcode.ps1</code>	locks the pcode library, which is the interpreter every session runs
<code>secure-psdir.ps1</code>	the directory privileged scripts are written into
<code>secure-reclaim.ps1</code>	creates the profile-reclaim store and locks it to SYSTEM
<code>secure-sysdirs.ps1</code>	takes Modify off the system directories that nothing writes

THE `secure-` FAMILY IS WHAT KEEPS SD's USERS OUT OF SD's OWN FILES. The data tree grants the `sdusers` group Modify, because every SD user needs it to use the database at all, and that grant is inherited everywhere. Each of these scripts takes it back off one thing that must not carry it.

The ones an administrator verb calls

Four SD verbs change the machine rather than the database, and each of them works by running one of these. **Prefer the verb.** It asks the questions that need asking, reports what happened in the product's own words, and refuses when it cannot act; the script does the work and assumes the caller knew what they were doing.

CALLED BY

<code>install-ssh.ps1</code>	<code>ssh.server install</code>
------------------------------	---------------------------------

CALLED BY

<code>remove-ssh.ps1</code>	<code>ssh.server remove</code> - takes the Windows OpenSSH server capability off the machine. The removal completes at the next restart
<code>ssh-firewall.ps1</code>	<code>remote.ssh on</code> <code>off</code> - scopes the shared Windows rule <code>OpenSSH-Server-In-TCP</code> rather than disabling it
<code>api-listener.ps1</code>	<code>remote.api on</code> <code>off</code> - writes or comments out the <code>APIPORT</code> line in <code>sd.conf</code>
<code>api-firewall.ps1</code>	<code>remote.api on</code> <code>local</code> - opens or restricts the API port
<code>sd-path.ps1</code>	<code>append.sd.path on</code> <code>off</code> - puts SD's program directory on the system PATH, or takes it off
<code>restart-sd.ps1</code>	offered by <code>remote.api on</code> and <code>off</code> , because the listener is only read at start-up

The verbs are covered in the SD Core for Windows administrator documentation, under *Remote access and the machine*.

The ones the uninstaller runs

<code>remove-sdaccounts.ps1</code>	takes away the Windows accounts SD created. Only runs if you ask for the data to be removed
<code>reclaim-profiles.ps1</code>	removes the Windows profiles SD had to leave behind at the time
<code>restore-sshonly.ps1</code>	puts every non-administrator SD account back into <code>sdsshonly</code> , reading the account register rather than anything local

`restore-sshonly.ps1` is the repair for a half-finished removal: an account that lost its deny rights but still exists would otherwise become an ordinary ssh login on the machine.

The ones SD runs for itself

These are launched by SD while it is running rather than by the installer. **Do not run them by hand**; they are listed because they appear in logs and in Task Manager.

<code>sd-elevate.ps1</code>	the unelevated half of an administrator session, called by SD's <code>ELEVATE</code> program with <code>-Start</code> , <code>-Run</code> or <code>-Stop</code> . Exit 0 done, 1 failed, 5 not elevated or elevation refused
<code>sd-elevate-helper.ps1</code>	the elevated half. <code>sd-elevate.ps1</code> launches it, which is where the UAC prompt appears, and it serves one SD session until that session ends
<code>micro-home.ps1</code>	gives the calling user a <code>micro</code> configuration home they can write to, and prints where it is. Run by the <code>EDIT</code> program before it launches <code>micro</code>
<code>reconcile-accounts.ps1</code>	removes register records whose Windows account has gone, and the account directory with them. Runs at every service start

A Windows process's token is fixed when it is created, so nothing can elevate a running process. That is why administrator work is done by a separate helper process rather than by an elevated `sd.exe`: SD stays unelevated for its whole life.

See also

Installation and the service covers what the installer puts on the machine and what an upgrade replaces. Remote access and the machine covers the four verbs that call seven of these scripts, and is the supported way to change any of those settings.

SD Basic - Restricted Commands

The statements and functions an ordinary program cannot compile, and what the compiler says when it tries.

Nothing on this page is available to an application: every name here needs a program compiled with `$internal`, which in turn needs an administrator in the `SDSYS` account. They are listed because they exist, because they appear in SD's own source, and because the errors they produce name something other than the real cause.

This document is separate so that it can be withheld. It links to nothing outside the administrator set. Where a page in another set is worth naming, it is named in words.

Italics mark something you supply, **bold** a word typed as it stands, and braces an optional part.

This page is generated from `BCOMP`'s own tables by `tools/mksyntax.py`, the same script that builds the User set's syntax card, and the two **partition** the roster: every name the compiler accepts is on exactly one of them, and the script refuses to write either page if that stops being true.

The three kinds, and how each one fails

(restricted)	a statement in <code>BCOMP</code> 's <code>restricted.statements</code> . An ordinary account gets "Unrecognised statement" , which at least names the right line
(internal)	a function in <code>BCOMP</code> 's <code>int.intrinsics</code> . The message names something else entirely : an unknown function is read as a matrix reference, so the complaint is "Matrix X is not referenced in a DIM statement" — reported at the last line of the program , nowhere near the call. With three or more arguments it is <i>"Right bracket not found where expected"</i> instead, because a matrix takes at most two subscripts
(no such thing)	in a table with no opcode behind it. It compiles for nobody — <code>\$internal</code> does not help

THE SECOND ROW IS THE ONE TO REMEMBER. If a function you are certain exists produces a complaint about a `dim` statement you never wrote, it does exist, and this account may not call it.

`$internal` needs both halves. `BCOMP` tests `kernel(K$INTERNAL, -1)` and `kernel(K$ADMINISTRATOR, -1)` — internal mode alone was enough until 13 Aug 2026, and it was not safe: internal programs are the only ones that may set the administrator flag, and `sd -internal` is not itself gated, so any account could have compiled a three-line program that granted itself administrator rights. That was demonstrated, not theorised.

A

<code>abort.cause</code>	<code>abort.cause()</code> (internal)
<code>add</code>	<code>add</code> <code>n</code> <code>to</code> <code>variable</code> (restricted)
<code>akclear</code>	<code>akclear</code> <code>file.var, index</code> <code>then</code> ... <code>else</code> ... (restricted)
<code>akdelete</code>	<code>akdelete</code> <code>file.var, index, key</code> <code>then</code> ... <code>else</code> ... (restricted)
<code>akenable</code>	<code>akenable</code> <code>file.var, index, state</code> (restricted)
<code>akmap</code>	<code>akmap</code> (<code>a, b</code>) (internal)
<code>akread</code>	<code>akread</code> <code>var</code> <code>from</code> <code>file.var, index, key</code> <code>then</code> ... <code>else</code> ... (restricted)
<code>akrelease</code>	<code>akrelease</code> <code>file.var, index, key</code> (restricted)
<code>akwrite</code>	<code>akwrite</code> <code>var</code> <code>to</code> <code>file.var, index, key</code> <code>then</code> ... <code>else</code> ... (restricted)
<code>analyse</code>	<code>analyse</code> (<code>a</code>) (internal)

B

<code>break.count</code>	<code>break.count()</code> (internal)
<code>breakpoint</code>	<code>breakpoint</code> <code>action, qualifier</code> (restricted)
<code>btree</code>	<code>btree</code> (<code>file.var, matrix</code>) — <code>matrix</code> is a <code>dim</code> med one-dimensional name (internal)

C

<code>callv</code>	<code>callv name (arg, ...) (restricted)</code>
<code>changed</code>	<code>changed(a) (internal)</code>
<code>como</code>	<code>como on name · como off · como name { on error ... } (restricted)</code>
<code>configure.file</code>	<code>configure.file file.var, key, value then ... else ... (restricted)</code>
<code>create.ak</code>	<code>create.ak file.var, index, dict.rec then ... else ... (restricted)</code>

D

<code>debug.info</code>	<code>debug.info(a, b) (internal)</code>
<code>debug.off</code>	<code>debug.off (restricted)</code>
<code>debug.on</code>	<code>debug.on (restricted)</code>
<code>debug.set</code>	<code>debug.set var {, qualifier} to value (restricted)</code>
<code>delete.ak</code>	<code>delete.ak file.var, index then ... else ... (restricted)</code>
<code>delete.common</code>	<code>delete.common name (restricted)</code>

E

<code>errmsg</code>	(no such thing) — in the statement table with no opcode behind it
<code>events</code>	<code>events(a, b) (internal)</code>
<code>expand.hf</code>	<code>expand.hf(a, b, c) (internal)</code>

F

<code>fcontrol</code>	<code>fcontrol(<i>a</i>, <i>b</i>, <i>c</i>)</code> (internal)
-----------------------	---

<code>find</code>	<code>find <i>x</i> in <i>arr</i> {, <i>occ</i>} setting <i>f</i> {, <i>v</i> {, <i>sv</i>}} then ... else ...</code> (internal)
-------------------	---

<code>formcsv</code>	<code>formcsv(<i>a</i>)</code> (internal)
----------------------	--

G

<code>getlocks</code>	<code>getlocks(<i>a</i>, <i>b</i>)</code> (internal)
-----------------------	---

<code>grpstat</code>	<code>grpstat(<i>a</i>, <i>b</i>)</code> (internal)
----------------------	--

I

<code>is.subr</code>	<code>is.subr(<i>a</i>)</code> (internal)
----------------------	--

<code>ismv</code>	<code>ismv(<i>a</i>)</code> (internal)
-------------------	---

K

<code>kernel</code>	<code>kernel(<i>a</i>, <i>b</i>)</code> (internal)
---------------------	---

<code>keyboard.input</code>	<code>keyboard.input <i>string</i></code> (restricted)
-----------------------------	---

L

<code>list.common</code>	<code>list.common()</code> (internal)
<code>load.object</code>	<code>load.object(<i>a</i>)</code> (internal)
<code>loaded</code>	<code>loaded(<i>a</i>)</code> (internal)
<code>login</code>	<code>login(<i>a</i>, <i>b</i>)</code> (internal)
<code>login.port</code>	<code>login.port(<i>a</i>, <i>b</i>)</code> (internal)
<code>logout</code>	<code>logout(<i>a</i>, <i>b</i>)</code> (internal)

M

<code>modify</code>	<code>modify <i>file.var</i>, <i>id</i></code> (restricted)
---------------------	--

O

<code>option</code>	<code>option(<i>a</i>)</code> (internal)
<code>ospath</code>	<code>ospath(<i>a</i>, <i>b</i>)</code> (internal)
<code>osrename</code>	<code>osrename(<i>a</i>, <i>b</i>)</code> (internal)

P

<code>pconfig</code>	<code>pconfig(<i>a</i>, <i>b</i>)</code> (internal)
<code>phantom</code>	<code>phantom()</code> (internal)

<code>prompt</code>	<code>prompt</code> <i>character</i> (internal)
---------------------	--

<code>pterm</code>	<code>pterm(<i>a</i>, <i>b</i>)</code> (internal)
--------------------	--

<code>pwcrypt</code>	<code>pwcrypt(<i>a</i>)</code> (internal)
----------------------	--

Q

<code>quit</code>	<code>quit</code> (restricted)
-------------------	---------------------------------------

R

<code>readpkt</code>	<code>readpkt()</code> (internal)
----------------------	--

<code>release.lock</code>	<code>release.lock</code> <i>n</i> (restricted)
---------------------------	--

<code>remove.token</code>	<code>remove.token</code> (restricted)
---------------------------	---

<code>removef</code>	<code>removef(<i>string</i> {, <i>occurrence</i>})</code> (internal)
----------------------	---

<code>reset.modes</code>	<code>reset.modes</code> <i>mask</i> (restricted)
--------------------------	--

<code>rewind</code>	<code>rewind</code> <i>file.var</i> <code>then</code> ... <code>else</code> ... (restricted)
---------------------	---

<code>run</code>	<code>run</code> { <i>file</i> } <i>record</i> { <i>options</i> } (restricted)
------------------	---

S

<code>scan</code>	<code>scan(<i>a</i> {, <i>matrix</i>})</code> — <i>matrix</i> is a <code>dim</code> med one-dimensional name (internal)
-------------------	--

<code>sdext</code>	<code>sdext(<i>a</i>, <i>b</i>, <i>c</i>)</code> (internal)
--------------------	--

<code>set.modes</code>	<code>set.modes</code> <i>mask</i> (restricted)
------------------------	--

<code>set.status</code>	<code>set.status</code> <i>n</i> (restricted)
<code>set.trigger</code>	<code>set.trigger</code> <i>file.var, mode, program {, modes}</i> (restricted)
<code>set.unassigned</code>	<code>set.unassigned</code> <i>var</i> (restricted)
<code>sortadd</code>	<code>sortadd</code> <i>key, data</i> (restricted)
<code>sortclear</code>	<code>sortclear</code> (restricted)
<code>sortdata</code>	<code>sortdata()</code> (internal)
<code>sortinit</code>	<code>sortinit</code> <i>keys, memory</i> (restricted)
<code>sortnext</code>	<code>sortnext(<i>matrix</i>)</code> — <i>matrix</i> is a <code>dim</code> med one-dimensional name (internal)

T

<code>testlock</code>	<code>testlock(<i>a</i>)</code> (internal)
-----------------------	---

U

<code>unload.object</code>	<code>unload.object</code> <i>class</i> (restricted)
----------------------------	---

V

<code>varset</code>	<code>varset</code> <i>var, value</i> (restricted)
---------------------	---

W

`watch`

`watch` *var* (***restricted***)

`writepkt`

`writepkt` *data* (***restricted***)

See also

Operating System Access covers the `sh` and `!` verbs, which are the other thing an ordinary account cannot reach. Encryption and the SDEXT interface explains why `sdext` is on this page and what it costs an application.

The User set's *SD Basic - Syntax* card carries everything an application may use, and the two partition the roster between them.

Features the Developers Could Not Test

The parts of SD Core for Windows that were built and reasoned about but never exercised, what is known about each, and what it would take to settle it.

Everything else in this documentation describes behaviour that was run and watched. **This page is the exception, and it exists so that the exception is visible in one place rather than scattered through the reference as footnotes.**

Nothing here is known to be broken. Each entry is something that **compiles, or exists, or follows from the source, and was never put under load or into the condition that would prove it.** Treat them as the list of things to pilot before an application depends on them.

This document is separate so that it can be withheld. It links to nothing outside the administrator set. Where a page in another set is worth naming, it is named in words.

Why it is in the administrator set. A reference page that keeps saying "*this part was not tested*" teaches an application programmer to distrust the whole book. An administrator deciding what to put into production needs exactly that information. So it is gathered here, and the user-facing pages state what SD does without qualifying every other paragraph.

How to read an entry

Known	what was actually run and observed
Not known	the specific gap — usually narrower than the heading suggests
To settle it	what would have to be done

Sessions and terminals

INTERACTIVE SD OVER SSH, AT A REAL TERMINAL

Known. An ssh session lands inside SD, and SD's terminal layer was watched driving a real Windows console.

Not known. Those two at once. Nobody has run an interactive session at a terminal *reached over ssh*, where the pseudo-terminal belongs to the ssh server rather than to the Windows console host. Screen handling, cursor positioning and the editing keys all go through that layer.

To settle it. One interactive session from a second machine, driving a full-screen operation and the arrow keys.

Locking and contention

SEMAPHORES UNDER CONTENTION

Known. The semaphores are exercised on every record lock, and two sessions competing for the same record ran through them at once without misbehaving.

Not known. No semaphore has ever been observed blocking. What is unmeasured is the waiting path, not the code path — the difference matters only under a load heavier than anything yet run.

To settle it. Enough concurrent sessions to make one wait, and a watch on what it does while it waits.

CONTENTION BETWEEN AN API SESSION AND A LOCAL ONE

Known. Two *local* sessions compete correctly: record, update and file locks are all reported against the right holder, a waiting read is released when the holder lets go, and a task lock refuses a second taker.

Not known. The same contest with one side arriving through the API server rather than at a terminal.

To settle it. An API client and a terminal session competing for one record.

TASK LOCKS TAKEN TWICE BY THE SAME SESSION

Known. From the source: taking a task lock you already hold succeeds, and one `unlock` releases it however many times you locked it — the ownership test is "*unowned or mine*", and `unlock` clears the slot outright.

Not known. It was **read rather than run**. No program has taken the same task lock twice and then released it once.

To settle it. Four lines of SD BASIC.

Application data

A REAL APPLICATION'S DATA

Known. SD creates, writes, reads and deletes files, records, indexes, select lists and sequential files, and the system files it bootstraps with are real ones.

Not known. No production application's data has been loaded into this port. Nothing here has met a file of hundreds of thousands of records, a deep dictionary, or a schema built over years by somebody else.

To settle it. Restore an existing account and run it.

Sockets

UDP AND ICMP

Known. TCP works: listening, connecting, accepting, reading, writing, the blocking and non-blocking modes, and the error codes.

Not known. The `0x00010000` and `0x00020000` flags are named in the documentation because they are **in the compiler**, not because a datagram was ever sent. No UDP or ICMP socket has been opened.

To settle it. A datagram to a listener and back.

Scheduled tasks

TASK SCHEDULER WITH AN ACCOUNT THAT `CREATE.ACCOUNT` MADE

Known. Scheduled SD jobs work when the task runs as a Windows account you already had. That is the case the design was built around.

Not known. Accounts that `create.account` creates are **denied interactive logon at this machine on purpose**, and whether Task Scheduler will accept one of them as the identity a task runs as has never been tried. It may refuse the credential outright.

To settle it. Create an account, point a scheduled task at it, and see whether the task page accepts it.

SD BASIC statements that compile but were never run

WHY NOT

<code>sendmail</code>	needs a mail relay configured
<code>chgphant()</code>	needs a phantom process to change
<code>ccall()</code>	needs a C function registered into the executable

Known. All three compile in an ordinary account.

Not known. What any of them does. Nothing else in the documentation depends on them.

Third-party editors

The key bindings documented for Microsoft Edit are the editor's own, read from its source rather than driven at a keyboard. SD installs it and calls it; it does not implement it. If a binding differs from what is written, the editor is right and the page is wrong.

What is NOT on this page, and why

Anything that was tested and failed is a defect, not a gap, and does not belong here — it is either fixed or it is a known issue.

Anything a reader might merely find surprising is not a gap either. The places where this port deliberately differs from OpenQM, ScarletDME or SD on Linux are documented as differences, in the pages that describe the feature. This page is only about what nobody has watched happen.

See also

Sessions and Locks covers the locking model that two of the entries above qualify. Installation and the service covers scheduled jobs and the account model behind the Task Scheduler entry. SD System Limits states which of its figures come from the source and which from a running system, and is the other page in this set that distinguishes the two.